

AI Test Plan Generator

Génération Intelligente de Plans de Test

Projet P5 — SIGMAXIS × ENSAM

| Automatiser la création de plans de test industriels grâce à un pipeline IA multi-agents, traçable et agnostique au fournisseur.

Contexte & Enjeux

"La création des plans de test est souvent manuelle, peu capitalisée et fortement dépendante de l'expertise individuelle." — Cahier des charges P5

Problèmes actuels

- Processus 100 % manuel, non capitalisé
- Expertise critique concentrée sur quelques personnes
- Traçabilité faible entre tests et exigences
- Pas de réutilisation inter-projets

Ce que le système résout

- Extraction automatique d'exigences depuis tout document
- Génération de plans et instructions testables
- Traceabilité complète (test → chunk → page source)
- Knowledge base réutilisable (général + projet)

Objectifs Couverts (Cahier des Charges)

#	Objectif	Statut
1	Générer automatiquement des plans de test	Implémenté
2	Capitaliser un knowledge base général et projet	Implémenté
3	Assurer la traçabilité tests ↔ documents sources	Implémenté
4	Produire des instructions de test exploitables	Implémenté
5	Deux niveaux de sortie : résumé et détaillé	Implémenté
6	Mode chatbot pour interaction et validation	Implémenté
7	Planifier et suivre l'exécution des tests	Implémenté
8	Agnostique au fournisseur IA (no vendor lock-in)	Implémenté

Architecture Globale du Système

```
flowchart TB
    subgraph SOURCES["Sources d'entrée"]
        PDF[PDF / DOCX / XLSX / MD]
    end
    subgraph INGESTION["Pipeline d'Ingestion"]
        L[Loaders Streamés] --> C[Chunker Hiérarchique]
        C --> E[Extracteur d'Exigences]
    end
    subgraph MEMORY["Mémoire Multi-Niveaux"]
        WM[Working Memory]
        EP[Mémoire Épisodique]
        SM[Mémoire Sémantique\nVector Store]
        GR[Graphe Cross-Documents\nTraçabilité]
    end
    subgraph BRAIN["Cerveau IA – Multi-Agents"]
        ORC[Orchestrateur\nSuperviseur]
        AGT[Agents Spécialisés\nx8]
    end
    subgraph OUTPUTS["Sorties"]
        PL[Plan de Test\nRésumé / Détaillé]
        TR[Matrice de\nTraçabilité]
        SC[Planning &\nGantt]
        CB[Copilot\nChatbot]
    end
    PDF --> INGESTION
    INGESTION --> MEMORY
    MEMORY <--> BRAIN
    BRAIN --> OUTPUTS
```

Le Cerveau IA : Topologie Multi-Agents

```
flowchart LR
    START([Entrée]) --> ORC
    ORC{Orchestrateur}
    ORC -->|1er passage| AN[Analyste\nDocumentaire]
    ORC -->|si besoin| EX[Extracteur\nExigences]
    ORC -->|plan vide| AR[Architecte\nTests]
    ORC -->|exigences prêtes| GN[Générateur\nCas de Test]
    ORC -->|cas générés| TR[Agent de\nTraçabilité]
    ORC -->|après traçabilité| RV[Réviseur\nQualité]
    ORC -->|approuvé| PL[Planificateur]
    RV -->|corrections critiques| ORC
    AN --> ORC
    EX --> ORC
    AR --> ORC
    GN --> ORC
    TR --> ORC
    PL --> ORC
    ORC -->|terminé| END([Plan Final])
    style ORC fill:#1a3a5c,color:#fff
    style END fill:#27ae60,color:#fff
    style START fill:#2980b9,color:#fff
```

Agents Spécialisés — Rôles & Modèles

Agent	Rôle	Tier LLM
Orchestrateur	Routage, détection de boucle, budget révisions	Fast
Analyste	Résumé corpus, détection de lacunes	Balanced
Extracteur d'Exigences	Map-reduce sur chunks, déduplication cosinus	Fast
Architecte de Tests	Stratégie, scope, critères d'entrée/sortie	Smart
Générateur de Cas	Étapes, critères d'acceptance, équipement	Balanced
Agent Traçabilité	Validation liens, coverage matrix	Balanced
Réviseur Qualité	Critique structurée, sévérités critical/major/minor	Smart
Planificateur	Jalons, affectation ressources, Gantt	Balanced
Copilot	Conversation grounded, mutations confirmées	Balanced

Chaque tier (`smart` / `balanced` / `fast`) peut pointer vers n'importe quel fournisseur via une variable d'environnement — aucun code à modifier.

Architecture Mémoire Multi-Niveaux

```
flowchart TD
    subgraph TIER1["Tier 1 – Working Memory (volatile, TTL)"]
        WM["Scratchpad de session\n• résumé corpus\n• draft courant\n• sorties d'outils"]
    end
    subgraph TIER2["Tier 2 – Mémoire Épisodique (ordonnée)"]
        EP["Journal de session\n• messages utilisateur\n• décisions agents\n• findings réviseur"]
    end
    subgraph TIER3["Tier 3 – Mémoire Sémantique (vectorielle)"]
        SM["Vector Store namespaces\nchunks:general\nchunks:{project}\nrequirements:{project}"]
    end
    subgraph TIER4["Tier 4 – Graphe Cross-Document (structurel)"]
        GR["NetworkX DiGraph\nDocument → Section → Chunk → Requirement → TestCase\nTraceLink typé : derives_from | covers | refines | contradicts"]
    end

    TIER1 -.-> TIER2
    TIER2 -.-> TIER3
    TIER3 -.-> TIER4

    note1["Swappable : \nTier 3 → Qdrant / Chroma / pgvector\nTier 4 → Neo4j / Memgraph"]
    style note1 fill:#f0f4f8,stroke:#2980b9,font-size:0.8em
```

Pipeline d'Ingestion — Streaming sur Grands Documents

```
flowchart LR
    subgraph IN["Fichier source (PDF/DOCX/XLSX/MD)"]
        F["10 000+ pages"]
    end

    subgraph LOAD["Loader (streaming)"]
        L1["PdfLoader\npypdf"]
        L2["DocxLoader\npython-docx"]
        L3["XlsxLoader\nopenpyxl read_only"]
        L4["MarkdownLoader"]
    end

    subgraph BLOCK["RawBlock stream\n(lazy iterator – 0(1 page))"]
        B["heading | prose\ntable | list_item\ncode | formula"]
    end

    subgraph CHUNK["Chunker Hiérarchique"]
        H["Pass 1 : Arbre de sections\n(numéros, titres, niveaux)"]
        T["Pass 2 : Packing token-bounded\n(prose) + blocs atomiques\n(tables, formules)"]
    end

    subgraph OUT["Sortie"]
        S["Sections avec offsets"]
        C["Chunks avec back-pointers"]
        E["Exigences extraites\n(LLM map-reduce)"]
    end

    F --> LOAD --> BLOCK --> H --> T --> OUT
    style F fill:#e8f4f8
```

Clé de scalabilité : le loader est un itérateur paresseux. Le chunker n'accumule jamais plus qu'une page en mémoire.

Traçabilité — La Colonne Vertébrale du Système

```
graph LR
    D["Document\nspec_v3.pdf"] -->|section| S["Section 4.2\nInterface Hydraulique"]
    S -->|chunk| C["Chunk ch_a3b9\np. 41-42"]
    C -->|derives_from| R["Requirement req_7f2a\n[performance] Pression max 250 bar"]
    R -->|covers| TC["TestCase tc_81c0\nTest pression nominale"]
    TC -->|step| ST["Step 3 : Appliquer 250 bar\nExpected: lecture ≤ 252 bar"]

    style D fill:#1a3a5c,color:#fff
    style R fill:#2980b9,color:#fff
    style TC fill:#27ae60,color:#fff
```

Chaque artefact est résolvable jusqu'au byte source :

- tc_81c0 couvre → req_7f2a dérive de → ch_a3b9 → section 4.2 → page 41

Mode Autonome (Fire-and-Forget)

```
sequenceDiagram
    participant U as Utilisateur
    participant ORC as Orchestrateur
    participant AG as Agents (x8)
    participant MEM as Mémoire

    U->>ORC: ingest(docs) + run(goal)
    loop Jusqu'à finish ou budget épuisé
        ORC->>ORC: Décision procédurale ou LLM
        ORC->>AG: route_to: analyst/extractor/architect/...
        AG->>MEM: register / retrieve
        AG-->>ORC: résultat typé
    end
    ORC->>ORC: reviewer → pas de critical → planner
    ORC-->>U: TestPlan + TestSchedule
```

Résultat : plan complet, révisé, planifié — sans interaction humaine.

Mode Interactif (Copilot Chatbot)

```
sequenceDiagram
    participant U as Ingénieur
    participant CP as CopilotAgent
    participant MEM as Mémoire
    participant LLM as LLM Gateway

    U->>CP: "Quels standards sont référencés ?"
    CP->>MEM: retrieve(query, project_id)
    MEM-->>CP: chunks + requirements + graph
    CP->>LLM: messages + grounding context
    LLM-->>CP: CopilotReply (message + citations)
    CP->>MEM: log episodic (user + assistant turn)
    CP-->>U: réponse + citations [spec_v3.pdf p.41]

    U->>CP: "Rédige un test pour REQ-4.2.1"
    CP-->>U: proposition + needs_confirmation=true
    U->>CP: "Confirme"
    CP->>MEM: register_test_case(...)
```

Agnostisme Fournisseur — Zéro Vendor Lock-in

```
flowchart TD
    subgraph AGENTS["Agents (ne connaissent que le protocole)"]
        A1[Orchestrateur] & A2[Générateur] & A3[Réviseur]
    end
    subgraph GW["LLMGateway Protocol"]
        P[complete / complete_structured\nstream / embed]
    end
    subgraph IMPL["Implémentation (LiteLLM)"]
        I1["claude-opus-4-1\n(Anthropic)"]
        I2["gpt-5\n(OpenAI)"]
        I3["gemini-2.5-pro\n(Google Vertex)"]
        I4["ollama/llama3.3\n(Local / Air-gap)"]
    end
    AGENTS --> GW
    GW --> IMPL

    ENV["Variable d'env :\nLLM_MODEL_SMART=gpt-5"]
    ENV -.->|change seule| IMPL

    style GW fill:#1a3a5c,color:#fff
    style ENV fill:#f39c12,color:#fff
```

Même logique pour le vector store : `SemanticStore` Protocol → InMemory | Qdrant | Chroma | pgvector.

Stack Technique

Orchestration & IA

- **LangGraph** — graphe d'agents avec état, checkpointing, interrupts
- **LiteLLM** — gateway universel 100+ providers
- **Pydantic v2** — modèles stricts, sorties structurées
- **Structured Output** — JSON Schema enforced sur chaque agent

Ingestion & Traitement

- **pypdf, python-docx, openpyxl** — loaders natifs streaming
- **NetworkX** — graphe de traçabilité (Neo4j-ready)
- **NumPy** — vector store in-memory (Qdrant-ready)

Qualité & Opérationnel

- `structlog` — observabilité structurée sur chaque agent
- `tenacity` — retries exponentiels sur les appels LLM
- `mypy` strict + `ruff` — lint & typage statique
- `hatchling` — packaging PEP 517

Structure du Code

```
src/ai_testplan_generator/  
├── config.py           ← Settings (pydantic-settings, env-driven)  
├── models/            ← 5 modèles Pydantic partagés  
├── llm/               ← Gateway Protocol + LiteLLM impl  
├── ingestion/         ← Loaders, Chunker, Extractor, Pipeline  
├── memory/            ← 4 tiers + MemoryManager  
├── knowledge/         ← General KB + Project KB  
├── prompts/           ← Bibliothèque de prompts centralisée  
├── agents/            ← 9 agents + State + Base  
├── graphs/            ← LangGraph autonomous + interactive  
├── pipelines/         ← Brain (racine), AutonomousPipeline, InteractivePipeline  
examples/  
├── run_autonomous.py  ← Demo fire-and-forget  
└── run_interactive.py ← Demo copilot REPL
```

~4 200 lignes · 40 modules · 0 erreur de compilation

Flux de Données Complet

```
flowchart TD
    subgraph A["1. Ingestion"]
        direction LR
        DOC[Fichiers] --> CHUNK[Chunks\n+sections]
        CHUNK --> EMBED[Embeddings\n→ Vector Store]
        CHUNK --> EXTRACT[Exigences\n→ Graphe]
    end
    subgraph B["2. Génération Autonome"]
        direction LR
        REQ[Exigences] --> ARCH[Stratégie\nde test]
        ARCH --> GEN[Cas de test\n×n exigences]
        GEN --> TRACE[Validation\ntraçabilité]
        TRACE --> REVIEW[Révision\nqualité]
        REVIEW --> PLAN[Planification\nressources]
    end
    subgraph C["3. Livrables"]
        direction LR
        PLAN --> OUT1[Plan Résumé\nou Détaillé]
        PLAN --> OUT2[Matrice de\nCouverture]
        PLAN --> OUT3[Gantt /\nJalons]
    end
    A --> B --> C
```

Avancement du Projet

Module	Statut	Commentaire
Ingestion PDF/DOCX/XLSX/MD	Complet	Streaming, 10k+ pages
Chunker hiérarchique	Complet	Overlap, atomique tables
Gateway LLM agnostique	Complet	LiteLLM + 3 tiers
Mémoire 4 niveaux	Complet	Interfaces + impl. ref.
9 agents spécialisés	Complet	Typés, structurés
Graphe LangGraph × 2 modes	Complet	Autonome + Interactif
Pipeline public (Brain)	Complet	Racine de composition
Backends persistants (prod)	Prochaine étape	Qdrant, Neo4j
API REST / Frontend	Hors scope actuel	Phase suivante
Tests unitaires	Prochaine étape	Mocks déjà prévus

Prochaines Étapes

```
gantt
  dateFormat YYYY-MM
  title Roadmap – Phases suivantes

  section Backends persistants
  Qdrant / Chroma adapter      :2025-05, 3w
  Neo4j traceability adapter   :2025-05, 3w
  SQLite episodic store        :2025-05, 2w

  section Observabilité & Qualité
  Tests unitaires & intégration :2025-06, 4w
  LangSmith / OTEL tracing      :2025-06, 2w

  section Exposition
  API REST (FastAPI)           :2025-06, 3w
  Interface utilisateur        :2025-07, 6w

  section Sécurité & Droits
  RBAC sur MemoryManager       :2025-07, 2w
  Chiffrement API Cloud        :2025-07, 2w
```

Merci

Périmètre livré à ce stade

Un pipeline IA complet, provider-agnostic, multi-agents, avec ingestion enterprise-scale, mémoire 4 niveaux et traçabilité complète.

Deux commandes pour démarrer :

```
pip install -e .  
python examples/run_autonomous.py spec.pdf norme.docx
```

Questions ?